



INSTITUTO DE ENGENHARIAS E CIÊNCIAS DO MAR

Ficha de Trabalho #4

▷ **Tópicos:**

- Análise de Complexidade de Algoritmos

Um dos pontos importantes na escolha de um algoritmo para a solução de um problema, para além da sua correção, é a sua complexidade. Discuta a questão da complexidade considerando o aspeto temporal.

1. Implemente um algoritmo eficiente que determina quantas ocorrências de um dado valor existem numa sequência de números inteiros. Faça a análise formal do melhor caso e do pior caso.
2. Implemente um algoritmo eficiente que determina se uma sequência de números inteiros é constituída por elementos distintos. Dito por outras palavras, a sequência não tem elementos repetidos. Faça a análise formal do melhor caso e do pior caso.
3. Faça a análise formal para os seguintes algoritmos, em termos da notação O .

(i) [fatorial de um número ($n!$)]

```
int fatorial(int n) {  
    int fat = 1;  
  
    for (i = 2; i <= n; i++)  
        fat = fat * i;  
    return fat;  
}
```

(ii) [soma de matrizes quadradas]

```
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        c[i][j] = a[i][j] + b[i][j];
```

(iii) [instrução soma++]

```
soma = 0;  
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        soma++;
```

(iv) [instrução soma++]

```
soma = 0;
for (i = 0; i < n; i++)
    for(j = 0; j < n*n; j++)
        soma++;
```

(v) [instrução soma++]

```
soma = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < i; j++)
        soma++;
```

(vi) [instrução soma++]

```
soma = 0;
k = 1;
for (i = 1; i <= n; i++) {
    for (j = 1; j <= k; j++)
        soma++;
    k = k * 2;
}
```

(vii) [instrução soma++]

```
soma = 0;
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n/i; j++)
        soma++;
}
```

4. Determine a complexidade do seguinte algoritmo para n par e potência de 2. Como sugestão, simule manualmente o algoritmo para $n = 4$, $n = 8$, $n = 16$ determinando o valor devolvido pelo algoritmo e o número de adições executadas.

```
int Algorithm1 (int n)
{
    sum = 0 ;
    for (i = 1; i <= n; i ++)
        for (j = n; j >= 1; j /= 2)
            sum += j;
    return sum;
}
```

5. Suponha que cada uma das expressões abaixo forneça o tempo de processamento $T(n)$ gasto por um algoritmo para resolver um problema de tamanho n . Selecione o termo dominante com o aumento de n e especifique a menor complexidade Big-Oh de cada algoritmo.

Expressão	Termo dominante	O(...)
$5 + 0.001n^3 + 0.025n$		
$500n + 100n^{1.5} + 50n\log_{10}n$		
$0.3n + 5n^{1.5} + 2.5n^{1.75}$		
$n^2\log_2n + n(\log_2n)^2$		
$3\log_8n + \log_2\log_2\log_2n$		
$100n + 0.01n^2$		
$2n + n^{0.5} + 0.5n^{1.25}$		
$100n\log_3n + n^3 + 100n$		
$0.003\log_4n + \log_2\log_2n$		

6. As instruções abaixo mostram algumas características da notação “Big-Oh” para as funções $f \equiv f(n)$ e $g \equiv g(n)$. Determine se cada instrução é verdadeira ou falsa e corrija a fórmula no último caso.

Instrução	Verdadeira ou Falsa	Caso for falsa então escreva a formula correta
$5n + 8n^2 + 100n^3 = O(n^4)$		
$5n + 8n^2 + 100n^3 = O(n^2\log n)$		
$2n^3 + n = O(n^3) \wedge n^3 = \Omega(n^3) \Rightarrow \theta(n^3)$		

7. Responder e comentar se é certo ou errado:

- A definição da notação Θ pode ser dada segundo a seguinte relação: Sejam f e h funções reais positivas da variável inteira n . Diz-se que $f = \Theta(h)$ quando existirem constantes $k_1, k_2 > 0$ e um valor inteiro de n_0 , tal que: $n > n_0 \Rightarrow k_1 \cdot h(n) \leq f(n) \leq k_2 \cdot h(n)$.
- Se a complexidade de melhor caso de um algoritmo for f , então o número de passos que o algoritmo efectua, qualquer que seja a entrada, é $\Omega(f)$.
- Se a complexidade de pior caso de um algoritmo for f , então o número de passos que o algoritmo efetua, qualquer que seja a entrada, é $\Theta(f)$.
- $2^{n+1} = O(2^n)$.
- $2^{2n} = O(2^n)$
- $5n^2 + 7n = \Theta(n^2)$
- $9n^3 + 3n = \Omega(n)$